Chedli Masmoudi

Tunis, Tunisia | +216 26 520 200 | chedli.masmoudi01@gmail.com | LinkedIn | GitHub

Final-Year Software Engineering Student — DevOps · Infrastructure as Code · Cloud & Platform Engineering

Seeking a 6-month Final-Year Internship (PFE) — Starting February 2027 — Open to relocation: Europe

Education

Fifth-Year Software Engineering Student | *INSAT* *Sep. 2022 – Present*

– Engineering degree (Master's equivalent), expected in 2027; coursework: DevOps, Distributed Systems and Cloud Infrastructure.

Technical Skills

Infrastructure as Code & Automation: Terraform, Ansible, Python, Bash
Cloud, Systems & Networking: Azure, AWS (fundamentals), Linux, VNet/NSG, Nginx, Nginx Ingress
CI/CD & GitOps: GitLab CI/CD, GitHub Actions, Jenkins, Argo CD, Git
DevSecOps & Governance: Policy-as-Code, OPA, Rego, Conftest, Azure RBAC, Key Vault, Linkerd mTLS
Containers & Platform Engineering: Kubernetes, AKS, Crossplane, Helm, Kustomize, HPA, Docker, Docker Compose
Observability & Reliability: Prometheus, Grafana, kube-prometheus-stack, ServiceMonitor
Software & Data: TypeScript/JavaScript, Node.js, NestJS, Express.js, FastAPI, Java/Spring Boot, PostgreSQL, Redis, MongoDB

Certification & Languages

HashiCorp Certified: Terraform Associate (004) | English: C1 (Professional) | French: C1 (Fluent)

Experience

DevSecOps Engineer Intern | *Cognira* *Jul. – Sep. 2026*

– **Architected and built a Policy-as-Code guardrail layer** to secure and govern Terraform-based Azure deployments. Authored **approximately 50 custom Rego deny/warn policies across 15 governance domains**, including access control, network exposure, sensitive-resource protection and cost governance; **Terraform plan JSON** is automatically evaluated against these policies through **Conftest/OPA before deployment**.
– **Built a Python reporting and decision engine** transforming Conftest JSON findings into reviewer-ready Terminal, Markdown and GitHub Actions reports, mapping violations to affected resources and producing **PASSED / REVIEW REQUIRED / BLOCKED / INCONCLUSIVE** verdicts for each Terraform plan evaluation.
– **Integrated the guardrail into the Terraform CI/CD pipeline with GitHub Actions**, automatically publishing validation verdicts and supporting **two execution modes**: non-blocking audit or **strict enforcement that fails the pipeline and prevents** `terraform apply` while active violations remain.
– **Designed an auditable temporary policy-exception workflow** using ticket references, exact **policy/resource/severity matching** and automated expiration checks; **approved and expired findings remain visible** to preserve full traceability.

Full Stack Engineer (Freelance) | *Collabios* *Nov. 2025 – Feb. 2026*

– **Architected an event-driven NestJS payment workflow** using Stripe Connect, PostgreSQL and Redis with idempotent asynchronous processing; **reduced payment-gateway fees by 40%** through an internal credit/refund mechanism while preserving transactional consistency.

Backend & DevOps Intern | *Billcom Consulting* *Jul. – Aug. 2025*

– **Reduced Docker image size for Spring Boot services by 60%** through multi-stage builds; built CI/CD pipelines with **Jenkins and GitHub Actions** and orchestrated multi-service environments with Docker Compose.
– **Developed a Java/Spring Boot REST API using Drools** to externalize telecom eligibility rules; achieved **85% code coverage with JUnit and Mockito**.

Selected DevOps & Platform Engineering Projects

Azure Subscription Factory — Terraform Automation & GitLab CI/CD

– **Built an Azure Subscription Factory with Terraform and GitLab CI/CD**: dashboard requests are persisted as YAML and passed by GitLab to a **dedicated Terraform module**, creating subscriptions under the **MCA billing scope** and associating them with **target Management Groups**; **validate → plan → manual approval → apply** enables plan review with service-principal authentication.

KubeForge — Kubernetes Self-Service Platform & GitOps | *GitHub*

– **Designed and developed a Kubernetes self-service platform** by **extending the Kubernetes API**: defined the declarative `DevEnvironment` resource using **Crossplane v2 (XRD-generated CRD)**, enabling developers to **trigger automated provisioning and lifecycle management of environments on a shared Kubernetes cluster through a single high-level YAML manifest**, without managing the underlying Kubernetes resources.
– **Designed and implemented a Crossplane Composition** using **Go templating**, transforming each `DevEnvironment` (XR) into **8 native Kubernetes resources** (Namespace, ResourceQuota, LimitRange, 2 Roles, ServiceAccount, RoleBinding, NetworkPolicy); **automated lifecycle management and drift correction** through continuous Crossplane reconciliation.
– **Developed a Python validator** checking `DevEnvironment` manifests (YAML syntax, XRD schema, platform rules); **integrated it into GitHub Actions CI triggered by Pull Requests**: noncompliant manifests cause a **required check to fail, blocking merge into** `main`; **configured Argo CD** to automatically synchronize validated `DevEnvironment` manifests merged into `main` with the Kubernetes cluster.
– **Developed and executed a 10-stage automated E2E test suite** verifying **actual platform behavior**: correct resource provisioning, quotas, RBAC permissions, network policies, configuration changes, drift correction and environment deletion; **10/10 stages passed**.

Task Manager — Azure, Kubernetes & GitOps Infrastructure | *GitHub*

– **Designed a Terraform-based Azure architecture** with two deployment strategies: a cost-conscious Linux VM using **Docker Compose and Ansible**, and a **scalable AKS cluster** for higher workloads.
– **Provisioned the Azure VM with Terraform and automated system and application configuration** through **idempotent Ansible playbooks**, deploying Nginx, backend services, PostgreSQL and Prometheus/Grafana.
– **Provisioned AKS with Terraform**, automating VNet/NSG configuration and the Helm-based deployment of Nginx Ingress, ArgoCD and kube-prometheus-stack; **collected application metrics with Prometheus through ServiceMonitor** and visualized them in **Grafana** dashboards.
– **Delivered a 5-minute code-to-deployment cycle** through an end-to-end GitOps workflow: CI builds SHA-tagged Docker images and updates deployment manifests, while ArgoCD performs pull-based deployment **without exposing cluster credentials to GitHub Actions runners**.
– **Diagnosed and eliminated an ArgoCD–HPA reconciliation loop** by excluding the `replicas` field from GitOps synchronization, allowing HPA to scale dynamically based on CPU/memory **without ArgoCD reverting runtime changes**.